

SafePay V1 — Backend Launch and Server Guide

1. Purpose and scope

This is the second deliverable governed by `dbsetup_cum_backend_master_guide_contract.md`. It explains how to apply the approved V12 seed through the existing Spring Boot/Flyway startup path, launch the backend safely on Windows, prove Oracle and application health, and establish the authentication context required for later API guides.

This guide does not:

- execute Flyway or start the backend on the user's machine;
- alter V1–V12, Java code, `application.properties`, database users or data;
- enable development email while its approved single-recipient override is absent;
- enable automatic settlement before the generated clearing-account ID is resolved;
- begin the Phase 2.1–2.12 implementation-understanding guide or API test guides.

The user performs every runtime/database action and confirms the results before the next contracted deliverable begins.

2. Canonical local runtime

Component	Approved local contract
Operating system	Windows
Java	Java 21 LTS; Eclipse Temurin is currently used, but Java 21 compatibility is the actual contract
Backend	Spring Boot 3.5.16
Packaging	Executable JAR with embedded Tomcat
Build tool	Maven Wrapper 3.9.16; a separate global Maven installation is unnecessary
Database	Oracle Database Free
Oracle host	localhost
Oracle listener port	1521
Oracle service	FREEPDB1, not CDB\$ROOT
Migration identity	SAFEPAY_OWNER
Runtime identity	SAFEPAY_APP
Backend port	8080 unless explicitly overridden
Backend base URL	http://localhost:8080
Approved local PWA origin	http://localhost:8000
API prefix	/api/v1
WebSocket endpoint	/ws

Project location:

C:\Users\Aditya Rao\Downloads\Training\Project\SafePay\backend

3. Launch sequence at a glance

Check Java, Maven Wrapper and Oracle

|

v

Verify SAFEPAY_OWNER and SAFEPAY_APP connect to FREEPDB1

|

v

Generate a private 256-bit JWT secret

|

v

Configure the required environment variables

|

v

FIRST START: all mutation schedulers disabled

|

+-- Flyway validates V1-V11

+-- Flyway applies V12 exactly once

+-- repeatable grants are validated/applied when required

+-- Hibernate validates mappings

+-- embedded Tomcat starts on 8080

|

v

Run health and read-only V12 database verification

|

v

Stop cleanly

|

v

SECOND START: normal protection/notification runtime enabled;

settlement and email still disabled

|

v

Run login, bearer-token, refresh-cookie and CSRF smoke tests

|

v

Optional, separately controlled settlement/WebSocket checks

4. Why the first startup is intentionally controlled

The default application configuration enables the protection scheduler and notification dispatcher. V12 contains no active `PROTECTED` timer, so the protection scheduler has nothing immediately eligible. It does contain a `PENDING` in-app notification, which the notification dispatcher can publish and mark `DE` `LIVERED` soon after startup.

For an exact first inspection of the newly inserted seed, temporarily use:

```
SAFEPAY_PROTECTION_SCHEDULER_ENABLED=false
SAFEPAY_NOTIFICATION_DISPATCHER_ENABLED=false
SAFEPAY_SETTLEMENT_PROCESSOR_ENABLED=false
SAFEPAY_OTP_EMAIL_ENABLED=false
```

This is a launch-time environment override only. It does not edit `application.properties`.

After the V12 counts and reconciliation checks pass, stop the server and restart with protection and notification enabled. Settlement remains disabled until its generated account ID is configured. OTP email remains disabled until the recipient-override implementation is separately approved and completed.

5. Prerequisite verification

Open a new Windows PowerShell terminal. These commands are read-only.

5.1 Java 21

```
java -version
javac -version
```

Both commands must report major version `21`. If Eclipse and PowerShell report different Java installations, correct both before proceeding:

- PowerShell must resolve a Java 21 `java.exe` and `javac.exe`.
- Eclipse/Spring Tools must use a Java 21 installed JRE for the SafePay launch configuration.
- Do not point `JAVA_HOME` at the `bin` directory; it must point at the JDK root.

5.2 Maven Wrapper

```
Set-Location 'C:\Users\Aditya Rao\Downloads\Training\Project\SafePay\backend'
.\mvnw.cmd -version
```

Expected essentials:

- Apache Maven 3.9.16;
- Java version 21;
- project base directory under the SafePay `backend` folder.

The first wrapper use may require internet access to download Maven. This project has already resolved its dependencies previously, so later launches can normally use the local Maven repository.

5.3 Oracle listener reachability

```
Test-NetConnection localhost -Port 1521
```

Expected: `TcpTestSucceeded : True`.

If false, list Oracle-related Windows services and start the correct local database/listener services through the normal Oracle/Windows service controls:

```
Get-Service *Oracle* | Select-Object Name, Status, DisplayName
```

Do not guess a service name or alter database configuration merely because one service is stopped.

5.4 SQL Developer connection checks

Use separate saved connections:

Suggested connection name	User	Host/port/service	Role
<code>safepay_owner_access</code>	<code>SAFEPAY_OWNER</code>	<code>localhost:1521/FREEDB1</code>	Default
<code>safepay_app_access</code>	<code>SAFEPAY_APP</code>	<code>localhost:1521/FREEDB1</code>	Default
Administrative connection	<code>SYSTEM</code>	<code>localhost:1521/FREEDB1</code>	Default, only when DBA work is actually required

Run this in both owner and runtime worksheets:

```
SELECT USER AS database_user,
       SYS_CONTEXT('USERENV', 'CON_NAME') AS container_name
FROM DUAL;
```

Expected:

- owner worksheet: `SAFEPAY_OWNER`, `FREEDB1`;
- runtime worksheet: `SAFEPAY_APP`, `FREEDB1`.

Stop if either connection reports `CDB$ROOT`, a different service, or the wrong user.

6. Environment-variable inventory

6.1 Required on every backend launch

Variable	Local value/format	Secret?	Purpose
<code>SAFEPAY_DB_URL</code>	<code>jdbc:oracle:thin:@//localhost:1521/FREEPDB1</code>	no	Shared Oracle JDBC location for Flyway and runtime.
<code>SAFEPAY_DB_OWNER_USERNAME</code>	<code>SAFEPAY_OWNER</code>	no	Privileged Flyway migration identity.
<code>SAFEPAY_DB_OWNER_PASSWORD</code>	Actual owner password	yes	Flyway authentication only.
<code>SAFEPAY_DB_APP_USERNAME</code>	<code>SAFEPAY_APP</code>	no	Restricted application datasource identity.
<code>SAFEPAY_DB_APP_PASSWORD</code>	Actual runtime password	yes	Application datasource authentication.
<code>SAFEPAY_JWT_SECRET_BASE64</code>	Base64 of at least 32 random bytes	yes	HMAC-SHA-256 signing and verification key.

The owner and runtime credentials must not be swapped. A successful owner connection does not prove the restricted runtime grants are correct.

6.2 Required/recommended for local browser operation

Variable	Local value	Why
<code>SAFEPAY_BROWSER_ORIGIN</code>	<code>http://localhost:8000</code>	Exact allowed browser and WebSocket origin. No wildcard is permitted.
<code>SAFEPAY_REFRESH_COOKIE_SECURE</code>	<code>false</code> for the current plain-HTTP local prototype	Allows the browser to return the refresh cookie over local HTTP. Set back to <code>true</code> for HTTPS.
<code>SERVER_PORT</code>	<code>8080</code> if explicitly set	Standard Spring port override; normally omitted because 8080 is the default.

An origin contains only scheme, host and optional port. Do not include a path, query, fragment, credentials or wildcard.

6.3 Scheduler controls

Variable	First controlled start	Normal local start
<code>SAFEPAY_PROTECTION_SCHEDULER_ENABLED</code>	<code>false</code>	<code>true</code>
<code>SAFEPAY_NOTIFICATION_DISPATCHER_ENABLED</code>	<code>false</code>	<code>true</code>
<code>SAFEPAY_SETTLEMENT_PROCESSOR_ENABLED</code>	<code>false</code>	Keep <code>false</code> until Section 15

These names use Spring Boot's environment-variable form of the existing property keys.

6.4 Conditional settlement variable

Variable	When required	Value
<code>SAFEPAY_SETTLEMENT_PROCESSOR_OUTBOUND_CLEARING_ACCOUNT_ID</code>	Only when settlement processor is enabled	Numeric ID queried from the V12 <code>SAFEPAY_OUTBOUND_CLEARING</code> account

The shorter name `SAFEPAY_OUTBOUND_CLEARING_ACCOUNT_ID` appears in a commented example inside `application.properties`. Because that property line is commented, the shorter variable alone does not configure the running application. Use the full Spring relaxed-binding name above unless a later approved configuration change wires the shorter alias.

6.5 Conditional Gmail variables — currently do not enable

Variable	Purpose
<code>SAFEPAY_OTP_EMAIL_ENABLED</code>	External override for <code>safepay.otp.email.enabled</code> ; must remain <code>false</code> now.
<code>SAFEPAY_OTP_EMAIL_USERNAME</code>	Development Gmail SMTP username.
<code>SAFEPAY_OTP_EMAIL_APP_PASSWORD</code>	Google App Password, never the normal Gmail password.
<code>SAFEPAY_OTP_EMAIL_FROM</code>	Valid sender address, normally the same development Gmail identity.

The approved future variables below are not currently consumed by the backend and setting them has no effect:

```
SAFEPAY_OTP_EMAIL_RECIPIENT_OVERRIDE_ENABLED
SAFEPAY_OTP_EMAIL_RECIPIENT_OVERRIDE
```

Until that adapter change is implemented, enabling email would send to the realistic-looking stored customer address. Keep email disabled.

7. Generate and validate the JWT secret

The earlier single-line static `RandomNumberGenerator.GetBytes(32)` form fails in Windows PowerShell/.NET versions that do not expose that static overload. Use this compatible form:

```
$jwtSecretBytes = New-Object byte[] 32
$jwtRandom = [System.Security.Cryptography.RandomNumberGenerator]::Create()
try {
    $jwtRandom.GetBytes($jwtSecretBytes)
    $env:SAFEPAY_JWT_SECRET_BASE64 = [Convert]::ToBase64String($jwtSecretBytes)
}
finally {
    $jwtRandom.Dispose()
}
```

Validate without printing the secret:

```
([Convert]::FromBase64String($env:SAFEPAY_JWT_SECRET_BASE64)).Length
```

Expected: `32` or greater.

To paste it into an Eclipse launch environment without displaying it in the console:

```
Set-Clipboard -Value $env:SAFEPAY_JWT_SECRET_BASE64
```

Paste into the value for `SAFEPAY_JWT_SECRET_BASE64`, then clear the clipboard:

```
Set-Clipboard -Value ''
```

Rules:

- generate one stable local secret and retain it securely for the current environment;
- changing it invalidates all existing access tokens;
- never store it in Git, Markdown, Java, properties, screenshots or test output;
- do not use a human phrase converted to Base64;
- do not share the value in chat.

8. Configure a PowerShell launch session

Use placeholders for passwords and replace them locally. Do not paste real passwords into documentation or chat.

```
Set-Location 'C:\Users\Aditya Rao\Downloads\Training\Project\SafePay\backend'

$env:SAFEPAY_DB_URL = 'jdbc:oracle:thin:@//localhost:1521/FREEPDB1'
$env:SAFEPAY_DB_OWNER_USERNAME = 'SAFEPAY_OWNER'
$env:SAFEPAY_DB_OWNER_PASSWORD = '<owner-password>'
$env:SAFEPAY_DB_APP_USERNAME = 'SAFEPAY_APP'
$env:SAFEPAY_DB_APP_PASSWORD = '<runtime-password>'

$env:SAFEPAY_BROWSER_ORIGIN = 'http://localhost:8000'
$env:SAFEPAY_REFRESH_COOKIE_SECURE = 'false'

$env:SAFEPAY_PROTECTION_SCHEDULER_ENABLED = 'false'
$env:SAFEPAY_NOTIFICATION_DISPATCHER_ENABLED = 'false'
$env:SAFEPAY_SETTLEMENT_PROCESSOR_ENABLED = 'false'
$env:SAFEPAY_OTP_EMAIL_ENABLED = 'false'
```

Generate the JWT value in the same terminal using Section 7. PowerShell environment assignments affect only this terminal and processes launched from it. Closing the terminal removes them.

Do not run a command that prints the entire environment. Verify only non-secret settings individually if needed:

```
$env:SAFEPAY_DB_URL
$env:SAFEPAY_DB_OWNER_USERNAME
$env:SAFEPAY_DB_APP_USERNAME
$env:SAFEPAY_BROWSER_ORIGIN
$env:SAFEPAY_PROTECTION_SCHEDULER_ENABLED
$env:SAFEPAY_NOTIFICATION_DISPATCHER_ENABLED
```

9. Configure Eclipse/Spring Tools

For a Spring Boot App or Maven Build launch configuration:

1. Open **Run Configurations**.
2. Select the SafePay configuration that launches `com.ofss.SafePayApplication`, or create one for the backend project.
3. Confirm the JRE is Java 21.
4. Open the **Environment** tab.
5. Choose **Append environment to native environment**.
6. Add the six always-required variables from Section 6.1.
7. Add `SAFEPAY_BROWSER_ORIGIN=http://localhost:8000`.

8. Add `SAFEPAY_REFRESH_COOKIE_SECURE=false` for local HTTP.
9. For the first controlled start, add the four disabled switches shown in Section 8.
10. Do not add Gmail credentials yet.
11. Apply the configuration.

For a Maven Build configuration:

- Base directory: the SafePay `backend` directory;
- Goal: `spring-boot:run`;
- JRE: Java 21;
- Environment: the same list above.

Keep passwords and JWT material only in the launch environment. Do not put them in Maven goals, VM arguments, `application.properties` or source control.

10. Pre-migration database checkpoint

Before the first V12 startup, run as `SAFEPAY_OWNER` :

```
SELECT installed_rank,  
       version,  
       description,  
       script,  
       installed_on,  
       success  
FROM "flyway_schema_history"  
ORDER BY installed_rank;
```

Expected before V12:

- successful versioned rows through V11;
- the repeatable grants migration is present/successful as applicable;
- no successful version `12` row yet.

Check seed markers:

```
SELECT
    (SELECT COUNT(*) FROM APP_USER
     WHERE email = 'priya.nair@gmail.com') AS priya_marker,
    (SELECT COUNT(*) FROM ACCOUNT
     WHERE account_number IN (
        'SAFEPAY_OUTBOUND_CLEARING',
        'SAFEPAY_OPENING_BALANCE_CONTROL')) AS system_account_markers,
    (SELECT COUNT(*) FROM PAYMENT_TRANSACTION
     WHERE transaction_reference LIKE 'SPV1-SHOWCASE-%') AS transaction_markers
FROM DUAL;
```

Expected before V12: all three values are zero.

If version 12 is already successful, do not delete its history row or rerun the migration. Follow the verification path instead.

11. Optional compile-only gate

This verifies Java production and test compilation without executing JUnit or starting Spring:

```
.\mvnw.cmd -DskipTests test-compile
```

Expected: **BUILD SUCCESS**.

This does not prove Oracle connectivity, apply V12, run JUnit or start the server.

12. First controlled startup: apply V12 and freeze background mutation

With the Section 8 environment in the same terminal:

```
.\mvnw.cmd spring-boot:run
```

Alternatively, run the Eclipse configuration containing the same values.

Do not start a second instance against the same port/schema during migration.

Expected startup stages

Look for these stages in order:

1. Spring Boot starts with application name **safepay**.
2. Hikari creates the runtime datasource.

3. Flyway connects with `SAFEPAY_OWNER`.
4. Flyway validates prior migration checksums.
5. Flyway applies `V12__safepay_v1_showcase_seed.sql` once.
6. V12's internal acceptance checks complete without `ORA-20xxx` errors.
7. Repeatable runtime grants are validated/reapplied only if their checksum requires it.
8. Hibernate validates entity mappings against `SAFEPAY_OWNER`; it does not create or update tables.
9. Embedded Tomcat listens on port 8080.
10. The log ends with `Started SafePayApplication`.

The server terminal must remain running while health/API calls are made.

Stop immediately if

- Flyway reports validation or checksum failure;
- V12 raises an application error;
- Hibernate schema validation fails;
- the runtime datasource cannot connect as `SAFEPAY_APP`;
- JWT configuration is missing/invalid;
- Tomcat cannot bind to the configured port.

Do not repeatedly restart after a partially understood migration failure. Preserve the first complete error and diagnose it before another attempt.

13. First health check

Open a second PowerShell terminal:

```
$safePayBaseUri = 'http://localhost:8080'  
Invoke-RestMethod -Method Get -Uri "$safePayBaseUri/actuator/health"
```

Expected:

```
status  
-----  
UP
```

Health is intentionally unauthenticated and does not expose detailed database information.

If the endpoint cannot be reached, check the server terminal first. If it returns `DOWN` or `503`, Oracle/runtime datasource health is not proven.

14. Post-V12 database gate before login

Before running login, use the read-only verification queries in:

[Documentation for SafePay/SafePay_V1_Data_Seed_Complete_Guide.md](#)

At minimum, confirm:

```
SELECT installed_rank, version, description, script, checksum, installed_on, success
FROM "flyway_schema_history"
WHERE version = '12';
```

Expected: exactly one successful row.

Then confirm the seed contribution counts, account-to-ledger reconciliation, reservation reconciliation, posting balance, clearing total, OTP safety and Risk Review rows from that guide.

Run these checks before authentication because a successful login legitimately creates an `AUTH_SESSION` and additional security audit evidence.

Do not treat startup as verified merely because Tomcat opened port 8080. The gate is:

```
Flyway 12 successful
+ Hibernate validation successful
+ health UP
+ seed acceptance/reconciliation queries successful
```

15. Resolve but do not yet enable outbound settlement

Run as `SAFEPAY_OWNER` or with an account that has the required read grant:

```
SELECT account_id,
       account_number,
       account_type,
       owner_user_id,
       currency_code,
       current_balance,
       reserved_amount,
       status
FROM ACCOUNT
WHERE account_number = 'SAFEPAY_OUTBOUND_CLEARING';
```

Expected contract:

- one row;

- owner is `NULL`;
- type is `OUTBOUND_CLEARING`;
- currency is `INR`;
- status is `ACTIVE`;
- current balance is ₹362,501.02 immediately after V12.

Record the generated numeric `account_id` privately. Do not guess it and do not use the opening-balance control account.

Keep:

```
SAFEPAY_SETTLEMENT_PROCESSOR_ENABLED=false
```

until the initial seed verification is complete.

When settlement demonstration is separately intended, configure both:

```
$env:SAFEPAY_SETTLEMENT_PROCESSOR_OUTBOUND_CLEARING_ACCOUNT_ID = '<queried-id>'
$env:SAFEPAY_SETTLEMENT_PROCESSOR_ENABLED = 'true'
```

Then restart. Startup validation must accept the configured account before the scheduler is considered operational. The V1 processor polls every second, uses a maximum batch of 25, retries after 5 seconds, 30 seconds and 1 minute, then routes exhaustion to `MANUAL_REVIEW`.

Enabling it can settle eligible `RELEASED` rows such as showcase ref 123. Take/retain your verified baseline before doing so.

16. Stop after the controlled migration pass

In the server terminal, press:

```
Ctrl+C
```

Wait for Spring/Hikari shutdown messages and the command prompt to return. Do not close the terminal forcibly during an active migration.

If SQL Developer has an uncommitted manual transaction, explicitly `COMMIT` or `ROLLBACK` it before closing its worksheet. Backend shutdown does not manage SQL Developer transactions.

17. Second startup: normal non-email runtime

For the functional server pass:

```
$env:SAFEPAY_PROTECTION_SCHEDULER_ENABLED = 'true'

$env:SAFEPAY_NOTIFICATION_DISPATCHER_ENABLED = 'true'

$env:SAFEPAY_SETTLEMENT_PROCESSOR_ENABLED = 'false'

$env:SAFEPAY_OTP_EMAIL_ENABLED = 'false'


.\mvnw.cmd spring-boot:run
```

Expected Flyway behavior now:

- V12 is recognized as already successful;
- V12 is not executed again;
- checksum validation succeeds;
- application startup proceeds to Hibernate validation and Tomcat.

The notification dispatcher may move the seeded pending notification to **DELIVERED**. That is an expected runtime lifecycle change, not seed corruption.

18. Authentication smoke test in PowerShell

Run this in a second terminal. It uses Priya's seeded local credentials and retains cookies in an in-memory PowerShell web session.

```

$safePayBaseUri = 'http://localhost:8080'
$safePayOrigin = 'http://localhost:8000'

$loginHeaders = @{
    'Origin' = $safePayOrigin
    'X-Correlation-ID' = 'launch-login-001'
}

$loginBody = @{
    loginIdentifier = 'priya.nair@gmail.com'
    password = 'SafePay@PNair101'
} | ConvertTo-Json

$loginHttpResponse = Invoke-WebRequest `
    -UseBasicParsing `
    -Method Post `
    -Uri "$safePayBaseUri/api/v1/auth/login" `
    -ContentType 'application/json' `
    -Headers $loginHeaders `
    -Body $loginBody `
    -SessionVariable safePayWebSession

$loginResult = $loginHttpResponse.Content | ConvertFrom-Json
$accessToken = $loginResult.accessToken

$loginResult | Select-Object tokenType, expiresAt, userId, authorities

```

Expected:

- HTTP 200;
- `tokenType` is `Bearer`;
- one nonblank access token;
- expiry is approximately 15 minutes after issue;
- authorities contain `CUSTOMER`;
- an `HttpOnly` `SAFEPAY_REFRESH` cookie is retained inside `$safePayWebSession`;
- response includes `X-Correlation-ID: launch-login-001`.

Do not print or share `$accessToken`, the refresh cookie or credentials.

Login is not read-only: it creates/rotates authentication state and records security evidence.

19. Bearer-token smoke test

```
$bearerHeaders = @{
    'Authorization' = "Bearer $accessToken"
    'Origin' = $safePayOrigin
    'X-Correlation-ID' = 'launch-beneficiaries-001'
}

$beneficiaries = Invoke-RestMethod `
    -Method Get `
    -Uri "$safePayBaseUri/api/v1/beneficiaries" `
    -Headers $bearerHeaders

$beneficiaries
```

Expected: HTTP 200 and only Priya's owned beneficiary data. No user/customer ID is supplied by the client; ownership comes from the JWT principal.

Negative smoke check:

```
try {
    Invoke-RestMethod `
        -Method Get `
        -Uri "$safePayBaseUri/api/v1/beneficiaries" `
        -Headers @{ 'Origin' = $safePayOrigin }
}
catch {
    $_.Exception.Response.StatusCode.value__
}
```

Expected: **401**, not a redirect or HTML login page.

20. CSRF token and refresh-cookie smoke test

Only the cookie-authenticated **POST /auth/refresh** and **POST /auth/logout** operations require CSRF protection. Normal bearer-authenticated business APIs do not require a CSRF token.

Obtain the token using the same cookie session:

```

$csrfResponse = Invoke-WebRequest `
    -UseBasicParsing `
    -Method Get `
    -Uri "$safePayBaseUri/api/v1/auth/csrf" `
    -Headers @{
        'Origin' = $safePayOrigin
        'X-Correlation-ID' = 'launch-csrf-001'
    } `
    -WebSession $safePayWebSession

$csrfResult = $csrfResponse.Content | ConvertFrom-Json
$csrfResult | Select-Object headerName, parameterName

```

Do not print the token value. Build the refresh headers dynamically from the server's declared header name:

```

$refreshHeaders = @{
    'Origin' = $safePayOrigin
    'X-Correlation-ID' = 'launch-refresh-001'
}
$refreshHeaders[$csrfResult.headerName] = $csrfResult.token

$refreshHttpResponse = Invoke-WebRequest `
    -UseBasicParsing `
    -Method Post `
    -Uri "$safePayBaseUri/api/v1/auth/refresh" `
    -Headers $refreshHeaders `
    -WebSession $safePayWebSession

$refreshResult = $refreshHttpResponse.Content | ConvertFrom-Json
$accessToken = $refreshResult.accessToken
$refreshResult | Select-Object tokenType, expiresAt, userId, authorities

```

Expected: HTTP 200, a fresh access token and a rotated refresh cookie retained by the same web session.

The refresh/logout request must contain:

- exactly one approved `Origin` header;
- the refresh cookie;
- the matching CSRF token header and CSRF cookie.

A missing/wrong origin or CSRF token should return 403.

21. Logout smoke test

Obtain a fresh CSRF token after refresh, because token/cookie lifecycle may rotate:

```
$csrfResponse = Invoke-WebRequest `
    -UseBasicParsing `
    -Method Get `
    -Uri "$safePayBaseUri/api/v1/auth/csrf" `
    -Headers @{
        'Origin' = $safePayOrigin
        'X-Correlation-ID' = 'launch-csrf-logout-001'
    } `
    -WebSession $safePayWebSession

$csrfResult = $csrfResponse.Content | ConvertFrom-Json

$logoutHeaders = @{
    'Authorization' = "Bearer $accessToken"
    'Origin' = $safePayOrigin
    'X-Correlation-ID' = 'launch-logout-001'
}

$logoutHeaders[$csrfResult.headerName] = $csrfResult.token

$logoutResponse = Invoke-WebRequest `
    -UseBasicParsing `
    -Method Post `
    -Uri "$safePayBaseUri/api/v1/auth/logout" `
    -Headers $logoutHeaders `
    -WebSession $safePayWebSession

$logoutResponse.StatusCode
```

Expected: 204. The refresh session is revoked and the server returns an expired/cleared refresh cookie.

22. Header, token and cookie rules for later API testing

Mechanism	When used	Rule
<code>Authorization: Bearer <token></code>	Every protected REST API	Access token is valid for 15 minutes and carries exact authorities.
<code>SAFEPAY_REFRESH</code> cookie	Refresh and logout only	HttpOnly, SameSite Strict, path <code>/api/v1/auth</code> ; not a general API authentication mechanism.
CSRF token + cookie	Refresh and logout POST only	Obtain from <code>/api/v1/auth/csrf</code> ; send using the returned header name.
<code>Origin</code>	Browser CORS and refresh/logout origin enforcement	Must exactly equal an approved configured origin.
<code>X-Correlation-ID</code>	Recommended on every request	<code>[A-Za-z0-9._:-]</code> , 1–64 characters; invalid/missing input is replaced with a generated UUID.
<code>Idempotency-Key</code>	Mutating payment/OTP/review operations that require it	Send exactly one header. Reuse only for a retry of the same logical request and same payload.
<code>Idempotency-Replayed</code>	Response header	Indicates a stored idempotent response was replayed.

Generate per-action identifiers in PowerShell:

```
$correlationId = [guid]::NewGuid().ToString()
$idempotencyKey = [guid]::NewGuid().ToString()
```

Do not reuse an idempotency key across different endpoints, actions, users or request bodies.

23. Browser CORS and cookie contract

Current browser controls are:

- explicit origin allow-list only;
- credentialed requests enabled;
- wildcard origins prohibited;
- methods `GET`, `POST`, `PUT`, `PATCH`, `DELETE`, `OPTIONS`;
- request headers limited to authorization, content type, origin, CSRF, correlation and idempotency headers;
- response headers exposing `X-Correlation-ID` and `Idempotency-Replayed`;
- stateless Spring Security; no HTTP session login;
- CSP `default-src 'none'; frame-ancestors 'none'` for API responses;
- frame denial, no-referrer policy and restrictive permissions policy.

For frontend calls that need the refresh cookie, JavaScript must use credentials, for example `credentials: 'include'`. The frontend must never read the HttpOnly refresh-token value.

If the frontend port changes, update `SAFEPAY_BROWSER_ORIGIN` to the exact new origin and restart the backend. Do not solve CORS failures with `*`.

24. WebSocket/STOMP launch contract

Handshake endpoint:

```
ws://localhost:8080/ws
```

The browser Origin must match `SAFEPAY_BROWSER_ORIGIN`. The STOMP `CONNECT` frame must contain exactly one native header:

```
Authorization: Bearer <current-access-token>
```

Allowed subscriptions:

Destination	Required authority
<code>/user/queue/notifications</code>	<code>CUSTOMER</code>
<code>/user/queue/transactions</code>	<code>CUSTOMER</code>
<code>/user/queue/risk-reviews</code>	<code>RISK_OFFICER</code>

All client STOMP `SEND` commands are rejected. SafePay WebSocket traffic is advisory; REST and Oracle remain authoritative. On reconnect, obtain a current access token, reconnect and reconcile data through REST.

Detailed WebSocket feature testing belongs in the later API/flow guides, not this startup smoke gate.

25. Gmail development settings — activation remains blocked

The backend is configured for `smtp.gmail.com:587`, SMTP authentication, required STARTTLS and finite five-second connection/read/write timeouts. The intended later activation process is:

1. use a dedicated development Gmail account;
2. enable Google 2-Step Verification;
3. create a Google App Password;
4. supply username, App Password and from-address only through environment variables;
5. enable the adapter only after the single-recipient override exists and is verified;
6. confirm every test OTP reaches only the approved developer inbox;
7. ensure no raw OTP or mail credential appears in logs, API responses or database fields.

Current stop condition:

```
SAFEPAY_OTP_EMAIL_ENABLED=false
```

Do not test real email delivery yet. The stored V12 customer addresses are presentation identities and must not be contacted.

26. Alternative packaged-JAR launch

After the controlled Flyway/launch path is proven, a packaged launch may be used:

```
Set-Location 'C:\Users\Aditya Rao\Downloads\Training\Project\SafePay\backend'  
.  
.\mvnw.cmd -DskipTests package  
java -jar '.\target\safepay-0.0.1-SNAPSHOT.jar'
```

The same environment variables are required. `-DskipTests` skips JUnit; this packaging command is not a test verification claim.

Do not run Maven and JAR server instances simultaneously on port 8080.

27. Optional full-suite regression gate

The backend's last user-confirmed complete suite contained 866 passing tests. To rerun it manually:

```
Set-Location 'C:\Users\Aditya Rao\Downloads\Training\Project\SafePay\backend'  
.  
.\mvnw.cmd test
```

The required database and JWT environment variables must be available. Surefire disables protection and notification schedulers so scheduled production work does not race integration fixtures.

Expected historical reference:

```
Tests run: 866, Failures: 0, Errors: 0, Skipped: 0  
BUILD SUCCESS
```

Treat the console output from the current run as authoritative. A different total must be explained by deliberate test additions/removals; `BUILD SUCCESS` alone is not evidence that 866 tests ran.

28. Safe shutdown

1. Stop issuing API requests.

2. Allow any intentionally invoked request to finish.
3. Press `Ctrl+C` once in the server terminal, or use the Eclipse stop control.
4. Wait for the application process to terminate.
5. Confirm port 8080 is no longer listening if necessary:

```
Test-NetConnection localhost -Port 8080
```

6. Explicitly commit or roll back independent SQL Developer transactions.
7. Close terminals containing secrets. Session-scoped environment values disappear with the terminal.

Do not end the Java process during Flyway migration or a deliberate financial mutation unless the process is irrecoverably hung and the resulting database state will be inspected before restart.

29. Startup and smoke-test diagnosis

Symptom	Likely cause	Correct first check
<code>java</code> / <code>javac</code> is not 21	PATH/JDK mismatch	Compare <code>java -version</code> , <code>javac -version</code> , Eclipse JRE.
Maven Wrapper cannot download	First-run network/proxy issue	Confirm network or use the already populated wrapper cache; do not replace project Maven versions casually.
<code>ORA-12514</code> / service unknown	Wrong Oracle service or listener registration	Confirm <code>FREEPDB1</code> , not <code>FREE</code> , <code>FREEDB1</code> or <code>CDB\$ROOT</code> .
<code>ORA-01017</code>	Wrong username/password	Test the same identity in SQL Developer without exposing credentials.
<code>ORA-00942</code> under runtime	Missing/wrong schema grant or wrong default schema	Confirm repeatable grants, <code>SAFEPAY_APP</code> , and <code>SAFEPAY_OWNER</code> default schema.
Flyway checksum mismatch	An applied migration was edited	Stop; compare the applied file/checksum. Never repair merely to hide an unauthorized edit.
V12 <code>-20300/-20301</code>	Canonical V11 roles/policy absent or inconsistent	Verify V11 and reference data.
V12 <code>-20302</code>	One or more V12 markers already exist	Determine whether V12 was manually/partially seeded; do not delete blindly.
V12 <code>-20310</code> – <code>-20318</code>	Seed count or reconciliation invariant failed	Preserve the exact first error and inspect the relevant V12 acceptance check.
<code>secretBase64</code> is required	JWT environment variable missing	Add <code>SAFEPAY_JWT_SECRET_BASE64</code> to the actual launch configuration.
Base64/256-bit JWT error	Malformed or too-short secret	Regenerate using Section 7; validation length must be at least 32 bytes.
Port 8080 already in use	Another server/process is active	Identify the existing listener; do not start duplicate SafePay instances.
Health endpoint is unreachable	Server did not finish startup or wrong port	Read the server console from the first exception.
Health is <code>DOWN</code> /503	Datasource/database health failure	Check Oracle listener, service, runtime credentials and grants.
Login returns 401	Wrong credential, locked/disabled user or stale data	Use exact seeded credential; inspect safe error and relevant user state.
Protected REST returns 401	Missing/expired/invalid JWT	Login/refresh and send the current bearer token.
Protected REST returns 403	Valid identity lacks required authority	Use the correct persona; never rename authority to <code>ADMIN</code> .

Symptom	Likely cause	Correct first check
Refresh/logout returns 403	Missing/wrong Origin or CSRF token/cookie pair	Re-fetch CSRF using the same web session and exact origin.
Browser reports CORS failure	Frontend origin differs from configured origin	Match scheme, host and port exactly; no wildcard.
Refresh works in tool but not browser	<code>Secure</code> cookie over local HTTP or missing credentials mode	Use local <code>SAFEPAY_REFRESH_COOKIE_SECURE=false</code> and browser credentials inclusion.
OTP delivery returns unavailable	Email intentionally disabled	Expected until the recipient override is implemented.
Settlement processor fails startup	Clearing ID absent/invalid	Query and configure the exact active ownerless INR outbound-clearing account ID.
Notification status changes after second start	Dispatcher processed pending V12 notification	Expected lifecycle behavior after enabling the dispatcher.
WebSocket CONNECT fails	Missing/expired STOMP bearer header or wrong Origin	Send exactly one native Authorization header and approved Origin.

Always diagnose the first meaningful exception rather than the final cascade.

30. Security and operational prohibitions

- Never commit database passwords, JWT secrets, access tokens, refresh tokens, Gmail App Passwords or raw OTPs.
- Never expose secret values in screenshots or console transcripts.
- Never launch Flyway using `SAFEPAY_APP`.
- Never run the application datasource as `SAFEPAY_OWNER` merely to avoid a grant error.
- Never edit or delete an applied Flyway migration/history row to force a rerun.
- Never enable Flyway clean against a valuable schema.
- Never disable ledger, audit, OTP, review or notification protection triggers for a launch test.
- Never enable SMTP against the realistic-looking V12 customer emails before the override exists.
- Never enable settlement using a guessed account ID.
- Never use a wildcard credentialed CORS origin.
- Never present simulated outbound clearing as live NPCI/RBI rail settlement or reversal.

31. User verification checklist

Environment

- ☐ `java -version` and `javac -version` report Java 21.

- ☐ Maven Wrapper reports Maven 3.9.16 and Java 21.
- ☐ Oracle listener port 1521 is reachable.
- ☐ Owner and runtime users both connect to `FREEPDB1`.
- ☐ Six required environment variables are present in the actual launch process.
- ☐ JWT Base64 decodes to at least 32 bytes.
- ☐ Browser origin is exact and refresh cookie is non-secure only for local HTTP.

First controlled start

- ☐ Protection, notification, settlement and email workers are disabled.
- ☐ Flyway validates V1–V11 and applies V12 once.
- ☐ No V12 acceptance check fails.
- ☐ Hibernate schema validation passes.
- ☐ `Started SafePayApplication` appears.
- ☐ `/actuator/health` returns `UP`.
- ☐ V12 Flyway history row is successful.
- ☐ Data Seed Complete Guide reconciliation queries pass.
- ☐ Generated outbound-clearing account ID is recorded safely.

Functional second start

- ☐ V12 is not rerun.
- ☐ Protection and notification processing are enabled deliberately.
- ☐ Settlement remains disabled unless its exact ID is configured.
- ☐ OTP email remains disabled.
- ☐ Seeded customer login returns a 15-minute bearer token and refresh cookie.
- ☐ Owned bearer-protected request succeeds.
- ☐ Missing bearer token returns 401.
- ☐ CSRF token plus approved Origin allows refresh.
- ☐ Logout returns 204 and revokes/clears refresh state.
- ☐ Backend shuts down cleanly.

32. Approval gate and next deliverable

The backend is ready for the next guide only after the user confirms:

1. V12 applied successfully through Flyway;
2. the seed reconciliation checks passed;
3. the backend starts with Hibernate validation successful;
4. health returns `UP`;
5. login, bearer authorization, refresh/CSRF and logout smoke tests work;

6. no SMTP message was sent to a seeded customer address;
7. any scheduler-induced changes were intentional and understood.

After that confirmation and explicit approval, work may begin on the Phase 2.1–2.12 Master Backend Understanding Guide. The separate phase-wise pure API test guides remain distinct deliverables and are not appended to this launch guide.